

# CSS (Cascading Style Sheets)

**Name: Saurav Khadka | Student ID: 250784206**

## 1. What is CSS?

CSS (Cascading Style Sheets) is a stylesheet language used to control the appearance and layout of HTML documents. While HTML provides the structure and content of a webpage, CSS is responsible for the visual presentation, including colours, fonts, spacing, positioning, animations, and responsive design.

CSS allows developers to separate content from presentation, making websites easier to maintain and update. By applying styles through CSS, developers can create visually appealing and consistent user interfaces across different devices and screen sizes.

### Core Features of CSS

- Selectors and Properties are used to select HTML elements and define how they should appear on a webpage.
- Typography Control allows developers to customise text by changing fonts, sizes, spacing, alignment, and font weight.
- Colours and Backgrounds help improve the visual appearance of a website through the use of colours, gradients, and background images.
- The Box Model is used to control the size of elements and the spacing around them using content, padding, borders, and margins.
- Layout Systems such as Flexbox and CSS Grid make it easier to organise and position content on a webpage.
- Responsive Design ensures that a website can adapt to different screen sizes and devices through flexible layouts and media queries.
- Transitions and Animations add smooth visual effects and movement to make webpages more interactive.
- Pseudo-classes and Pseudo-elements allow styles to be applied to specific element states or to create additional visual effects.
- CSS Variables provide a way to store reusable values, making stylesheets easier to maintain and update.
- Accessibility Features help create websites that are easier to use by supporting better focus indicators, contrast, and motion settings.

## **2. Ways to implement CSS**

### **2.1 Inline CSS**

Applied directly on an element with the `style` attribute. Affects only that element. Used for very small, one-off tweaks or quick testing and not for production scale. It is fast to apply; overrides many other styles due to cascade specificity but also mixes content and presentation due to which it is hard to maintain, not reusable and inflates HTML.

### **2.2 Internal CSS**

Placed inside a `<style>` element within the `<head>` section of an HTML document. The styles defined apply only to that specific webpage. Internal CSS is useful for single-page websites, prototypes, or situations where a page requires unique styling that will not be reused elsewhere. One advantage is that all styling is kept within the same file, making it easier to manage small projects without creating additional files. It also avoids the need for a separate stylesheet request. However, the styles cannot be reused across multiple pages, and as a website grows, the HTML file can become cluttered and harder to maintain.

### **2.3 External CSS**

Stored in a separate .css file and linked to an HTML document using the `<link>` element. This is the most common approach for modern websites because it keeps styling separate from content. It is particularly useful for multi-page websites where the same styles need to be reused across different pages. The main advantages are improved maintainability, cleaner HTML code, and the ability for browsers to cache stylesheets, which can improve performance. However, it requires an additional file request and may require better file organisation as projects become larger.

### 3. CSS Demonstration – Cafe Menu Webpage

Live Demo Link: <https://saurav09-ops.github.io/CSS/>



Figure 1: Cafe menu Web page(CSS-DEMO)

For my CSS demonstration, I created a café menu webpage. The project uses HTML for structure and CSS for presentation. The objective was to transform a basic menu into a visually appealing and responsive webpage using modern CSS techniques.

#### 3.1 How the Webpage Works and What It Demonstrates

The webpage displays a café menu using a centred card-style layout positioned against a linear gradient background. CSS is used to control the visual appearance of the page through custom typography, colours, spacing, alignment, and responsive design techniques. Google Fonts are used to improve the overall presentation, while Flexbox is used to organise menu items and footer content consistently. Responsive features such as the `clamp()` function and media queries help ensure that text and layout elements adapt appropriately to different screen sizes. This demonstration shows how CSS can transform a basic HTML structure into a visually appealing, readable, and mobile-friendly webpage without changing the underlying content.

### 3.2 Features Implemented

The webpage includes:

- A linear gradient background to create a more visually appealing design.
- Custom typography using Google Fonts, including Pacifico for the main heading and Josefin Sans for the body content.
- A centred card-style layout that places the menu content in a clean and focused container.
- Responsive font sizing using the `clamp()` function to improve readability across different screen sizes.
- Flexbox layouts for menu items and footer content, providing consistent alignment and spacing.
- Consistent spacing using margins and padding to separate content into clear sections.
- Social media icons provided through Font Awesome to enhance the footer design.
- Responsive sizing using modern CSS functions such as `min()` to control the width of the menu card.
- Mobile-friendly adjustments for smaller screen sizes using media queries.
- A clean and organised visual hierarchy using different heading sizes, borders, and typography styles.

### 3.3 CSS Techniques Used

The following CSS techniques and properties were used throughout the project:

- An external stylesheet was used to separate presentation from HTML content. This improves maintainability and allows styles to be managed from a single file.
- A universal selector (\*) was used to reset default browser margin and padding values and apply `box-sizing: border-box`. This helps ensure consistent sizing and spacing across all elements.
- Google Fonts were imported to improve typography. The Pacifico font was applied to the main heading to create a distinctive visual identity, while Josefin Sans was used throughout the page to improve readability.
- A linear gradient background was applied to the body element to create visual depth and make the menu card stand out against the page background.
- Flexbox was used to centre the menu card within the page and to align menu items and footer content. This creates a cleaner and more responsive layout.
- The `clamp()` function was used for responsive typography. This allows text sizes to scale smoothly between minimum and maximum values based on screen size.
- The `min()` function was used to control the width of the menu card, helping the layout remain responsive while preventing the content from becoming too wide on larger screens.
- Margins, padding, borders, and border-radius properties were used to improve spacing, readability, and overall visual presentation.
- Font Awesome icons were integrated into the footer to demonstrate the use of external CSS libraries and improve the appearance of social media links.

- A media query was implemented for screens smaller than 576 pixels. This adjusts spacing and layout behaviour to improve usability on mobile devices.
- Colour, typography, spacing, layout, and responsive design principles were combined to create a professional and user-friendly interface without modifying the underlying HTML structure.

#### **4. Reflection**

Through this project, I gained practical experience applying CSS to improve the appearance and usability of a webpage. I learned how typography, colours, spacing, Flexbox layouts, and responsive design techniques can work together to create a professional user experience. This activity strengthened my understanding of how CSS enhances the presentation of web content while maintaining a clear separation between structure and design.